



Module 7 — Tests E2E avancés, Fixtures et Environnements

🎯 **Objectif du module** Maîtriser le niveau "Expert" : Automatiser un flux complet en **POM strict** (zéro sélecteur technique dans le test), gérer les données via **Fixtures** et configurer le "robot" Cypress via le fichier de **Config**.



Architecture et Organisation du Projet

L'étudiant doit organiser ses fichiers exactement comme ceci :

Plaintext

```
cypress-formation/
├── cypress/
│   ├── fixtures/
│   │   └── utilisateur.json          # Chemin : cypress/fixtures/utilisateur.json
│   ├── e2e/
│   │   └── module-7-tests-avances/  # Chemin : cypress/e2e/module-7-tests-avances/
│   │       ├── exemples/
│   │       │   ├── 01_demo_hooks.cy.js
│   │       │   ├── 02_tunnel_pom.cy.js
│   │       │   └── 03_demo_task.cy.js
│   │       └── exercices/
│   │           └── exercice_synthese.cy.js
│   ├── pages/                      # Chemin : cypress/pages/
│   │   ├── LoginPage.js
│   │   ├── ProductsPage.js
│   │   └── CheckoutPage.js
│   └── cypress.config.js            # RACINE DU PROJET
```



1 Configuration Avancée : `cypress.config.js`



Fichier : `cypress.config.js`

JavaScript

```
const { defineConfig } = require("cypress");

module.exports = defineConfig({
  e2e: {
    baseUrl: 'https://www.saucedemo.com',
    viewportWidth: 1280, // Taille d'un écran PC standard
    viewportHeight: 720,

    setupNodeEvents(on, config) {
      // TASK : Talkie-walkie entre le navigateur et le PC
      on('task', {
        alerteTerminal(message) {
          console.log("LOG NODE.JS : " + message);
        }
      });
    }
  }
});
```

```

        return null;
    }
    });
  },
},
});

```

⚙️ Étape 1 : Comprendre la structure par défaut

L'étudiant doit d'abord comprendre les trois blocs principaux du fichier que tu as partagé :

1. `const { defineConfig } = require("cypress") ;` : L'import de l'outil de configuration.
2. `e2e: { ... }` : Le bloc dédié aux tests de bout en bout.
3. `setupNodeEvents(on, config)` : L'emplacement pour les fonctionnalités avancées (comme les Tasks).
4. **Pourquoi le Viewport ?** Pour éviter que des éléments soient cachés ou empilés comme sur un téléphone.
5. **Pourquoi la Task ?** Pour faire des actions que le navigateur interdit (ex: supprimer un fichier).

⚙️ Étape 2 : L'intégration finale (Code à copier)

Voici comment tu dois présenter le fichier finalisé à l'étudiant. On part de ton squelette et on injecte les réglages du Module 7.

⚙️ Étape 3 : Explication de l'évolution (Pédagogie)

Pour que l'étudiant comprenne la transition, tu peux lui expliquer ainsi :

- **Avant (Squelette vide)** : "C'était une coquille vide qui utilisait les réglages par défaut de Cypress (écran petit, pas d'URL favorite)."
- **Après (Config Module 7)** : "On a personnalisé le robot. Maintenant, il sait sur quel site aller par défaut (`baseUrl`), il a une vue confortable (`viewport`) et il peut communiquer avec notre ordinateur (`task`)."

2 Les Hooks : Cycle de vie 🔄

 **Fichier** : `cypress/e2e/exemples/exemples/01_demo_hooks.cy.js`

JavaScript

```

describe('Maîtrise des Hooks', () => {
  beforeEach(() => {
    // BONNE PRATIQUE : Reset complet avant chaque test
    cy.clearCookies(); // Vide les cookies 🍪
    cy.clearLocalStorage(); // Vide le panier/session 📦
  });
});

```

```

    cy.visit('/');
  });

  it('Premier test propre', () => {
    cy.log("Je démarre sur une page vierge");
  });
});

```

Les Hooks Cypress : Le Cycle de Vie du Test

En entreprise, on n'écrit jamais un test isolé. On en écrit des centaines. Les hooks permettent d'automatiser les actions répétitives (nettoyage, connexion, log) pour garantir que chaque test démarre dans un état "sain".

Hook	Quand s'exécute-t-il ?	Application en Entreprise 
<code>before(() => { ... })</code>	Une seule fois , avant le tout premier test du fichier.	Préparer des données lourdes (ex: créer un utilisateur via une API) qui serviront à tous les tests suivants.
<code>beforeEach(() => { ... })</code>	Avant chaque test (it).	Le plus utilisé. Aller sur l'URL, vider les cookies ou le localStorage pour éviter que les restes du test précédent ne fassent échouer le suivant.
<code>afterEach(() => { ... })</code>	Après chaque test (it).	Envoyer un log de succès/échec ou réinitialiser un état spécifique dans l'application.
<code>after(() => { ... })</code>	Une seule fois , après le tout dernier test du fichier.	Nettoyer les données créées (ex: supprimer l'utilisateur de test en base de données) pour laisser l'environnement propre.

Pourquoi est-ce vital en entreprise ? (Avantages)

1. **Fiabilité (Anti-Flakiness) :** En utilisant `beforeEach` pour vider les cookies et le panier, vous garantisiez que le Test B ne plante pas parce que le Test A a laissé un produit dans le panier.
2. **Gain de temps (DRY - Don't Repeat Yourself) :** Au lieu d'écrire `cy.visit('/')` dans 50 tests, vous l'écrivez une seule fois dans le `beforeEach`.
3. **Maintenance simplifiée :** Si l'URL de votre site de test change, vous ne la modifiez qu'à un seul endroit (le hook ou la config).

Choses à éviter (Pièges de débutants)

- **Ne pas mettre d'assertions (`should`) dans les hooks :** Les hooks servent à **préparer**, les `it` servent à **tester**. Si votre `beforeEach` échoue, tous vos tests seront bloqués.

- **Abuser du before** : Si vous mettez trop d'étapes dans le `before` (qui ne s'exécute qu'une fois), un test peut modifier l'état et faire rater tous les suivants. **Privilégiez toujours le `beforeEach`** pour une isolation maximale.

2 Bis : Les Fixtures (La source de données)

Avant de lancer un tunnel complexe, nous isolons nos données (identifiants, noms) dans un fichier indépendant. Cela permet de ne jamais coder d'informations sensibles ou variables directement dans le test.

 **Fichier : `cypress/fixtures/utilisateur.json`**

```
{
  "user": "standard_user",
  "pass": "secret_sauce",
  "prenom": "Jean",
  "nom": "Dupont"
}
```

3 Le Tunnel 100% POM (Zéro sélecteur technique dans le test)

A. Les Pages (Le "Comment")

 **Fichier : `cypress/pages/ProductsPage.js`**

```
class ProductsPage {
  ajouterSacADos() {
    cy.get('[data-test="add-to-cart-sauce-labs-backpack"]').click();
  }
  ouvrirPanier() {
    cy.get('.shopping_cart_link').click();
    cy.get('[data-test="checkout"]').click();
  }
}
export default new ProductsPage();
```

 **Fichier : `cypress/pages/CheckoutPage.js`**

```
class CheckoutPage {
  remplirInfos(prenom, nom, cp) {
    cy.get('[data-test="firstName"]').type(prenom);
    cy.get('[data-test="lastName"]').type(nom);
    cy.get('[data-test="postalCode"]').type(cp);
    cy.get('[data-test="continue"]').click();
  }
  finaliserAchat() {
    cy.get('[data-test="finish"]').click();
  }
}
```

```
export default new CheckoutPage();
```

B. Le Test (Le "Quoi" - Métier uniquement)

C'est ici que la magie opère : on appelle le fichier de données via `cy.fixture()`, puis on injecte ces données dans nos pages POM.

 **Fichier : cypress/e2e/module-7-tests-avances/exemples/02_tunnel_pom.cy.js**

```
import loginPage from '../pages/LoginPage';
import productsPage from '../pages/ProductsPage';
import checkoutPage from '../pages/CheckoutPage';

describe('Tunnel Expert SauceDemo', () => {
  beforeEach(() => {
    cy.visit('/');
  });

  it('Flux complet avec POM et Fixtures', () => {
    // 1. On appelle le fichier utilisateur.json
    cy.fixture('utilisateur').then((data) => {

      // 2. On utilise 'data' pour remplir les méthodes du POM
      loginPage.login(data.user, data.pass);

      productsPage.ajouterSacADos();
      productsPage.ouvrirPanier();

      // On pioche le prénom et le nom dans la fixture
      checkoutPage.remplirInfos(data.prenom, data.nom, "75000");

      checkoutPage.finaliserAchat();

      // 3. Vérification finale
      cy.get('.complete-header').should('contain', 'Thank you');
    });
  });
});
```

Détails

1. **L'asynchronisme** : La commande `cy.fixture()` ne donne pas les données instantanément. Le bloc `.then((data) => { ... })` est obligatoire pour dire à Cypress : *"Une fois que tu as fini de lire le fichier JSON, exécute la suite avec ces données"*.
 2. **La portée** : La variable `data` n'existe qu'à l'intérieur des accolades du `.then()`. Si tu essaies d'utiliser `data.user` en dehors, le test plantera.
-

Exercice de Synthèse (À donner à l'étudiant)

 **Chemin :** `cypress/e2e/module-7-tests-avances/exercices/exercice_synthese.cy.js`

Consignes :

1. Configure ton viewport en 1280x720.
2. Dans un `beforeEach`, visite le site et vide le **LocalStorage**.
3. Utilise la **Fixture** `utilisateur.json` pour te connecter via le **POM**.
4. Utilise une **Assertion Conditionnelle** (via un `if` sur le `body`) pour vérifier si le bouton "Finish" est visible avant de cliquer.
5. Vérifie à la fin que le cookie `session-username` existe bien.

CORRECTION de l'exercice de synthèse

 **Fichier :** `cypress/e2e/module-7-tests-avances/exercices/exercice_synthese.cy.js`

JavaScript

```
import loginPage from '../../../pages/LoginPage';
import productsPage from '../../../pages/ProductsPage';
import checkoutPage from '../../../pages/CheckoutPage';

describe('Correction Exercice de Synthèse', () => {

  beforeEach(() => {
    cy.clearLocalStorage();
    cy.visit('/');
  });

  it('Réalisation du flux robuste', () => {
    cy.fixture('utilisateur').then((user) => {
      // 1. Connexion POM
      loginPage.login(user.user, user.pass);

      // 2. Actions POM
      productsPage.ajouterSacADos();
      productsPage.ouvrirPanier();
      checkoutPage.remplirInfos(user.prenom, user.nom, "75000");

      // 3. Assertion Conditionnelle (Le IF sur le Body)
      cy.get('body').then(($body) => {
        if ($body.find('[data-test="finish"]').length > 0) {
          cy.task('alerteTerminal', 'Bouton Finish détecté, je
finalise.');
```

```
          checkoutPage.finaliserAchat();
        }
      });

      // 4. Vérification finale et Cookies
      cy.get('.complete-header').should('be.visible');
      cy.getCookie('session-username').should('exist');
    });
  });
});
```

```
});  
});
```

Rappel des Bonnes Pratiques

- **Viewport** : Fixez-le toujours pour avoir la même vue que vos collègues.
 - **Hooks** : Le `beforeEach` est votre meilleur ami pour garantir que le test 2 ne plante pas à cause du test 1.
 - **POM** : Si vous écrivez `cy.get` dans un test, vous faites une erreur. Créez une méthode dans votre Page.
-

Conclusion du Module 7

À l'issue de ce module, l'étudiant a franchi un cap majeur. Il ne se contente plus d'écrire des tests, il conçoit une **stratégie d'automatisation robuste**.

Ce qu'il maîtrise désormais :

1. **L'industrialisation (Fixtures & Config)** : Il sait séparer les données du code et configurer son environnement de test (BaseURL, Viewport) pour qu'il soit identique partout.
2. **Le POM "Pure Souche"** : Il a compris que la lisibilité d'un test dépend de l'absence de sélecteurs techniques dans ses fichiers `.cy.js`. Il sait transformer un besoin métier en méthodes de Page.
3. **La maîtrise de la session** : Il sait manipuler les Cookies et le LocalStorage pour simuler des états complexes sans perdre de temps.
4. **L'ouverture vers le système (Tasks)** : Il a compris comment sortir des limites du navigateur pour interagir avec sa machine via Node.js.
5. **La robustesse** : Grâce aux assertions conditionnelles, il sait gérer l'imprévisibilité d'une application web (popups, bannières).